

# Dòng dữ liệu chi tiết: Một mẫu ECG 12 × 1000 đi từ DMA vào đến class output

Tài liệu này theo dõi một mẫu ECG đơn lẻ — vector kích thước (12 lead × 1000 mẫu thời gian) — đi qua toàn bộ pipeline FPGA `ITM_Top_v3`, mô tả từng tín hiệu, từng state FSM, từng substep, ánh xạ với phép tính PyTorch tương đương. Mục đích: người đọc nắm được **mọi thao tác phần cứng** thực sự xảy ra để tạo ra 5 logits ở cuối.

## 0. Ký hiệu và quy ước

### 0.1 Tín hiệu top-level chính

| Tín hiệu                                                                                        | Hướng | Width     | Ý nghĩa                                   |
|-------------------------------------------------------------------------------------------------|-------|-----------|-------------------------------------------|
| <code>clk</code>                                                                                | in    | 1         | Clock 100 MHz (period 10 ns)              |
| <code>rst</code>                                                                                | in    | 1         | Async reset, active high                  |
| <code>start</code>                                                                              | in    | 1         | Pulse 1 cycle để khởi động phase hiện tại |
| <code>T_MAX, T_ENC, T_GAP</code>                                                                | in    | 10        | Số timestep                               |
| <code>CH_IN, CH_OUT, CH_M, DT_RANK</code>                                                       | in    | 4         | Cấu hình block hiện tại                   |
| <code>enc_mode, head_mode, cascade_mode, need_pool</code>                                       | in    | 1         | Chế độ                                    |
| <code>dma_write_en, dma_target[1:0], dma_addr[14:0], dma_wdata[255:0]</code>                    | in    | —         | DMA write từ host                         |
| <code>done_encoder, done_phase1, done_inception, done_mamba, done_all, done_gap, done_fc</code> | out   | 1         | Pulse báo từng phase xong                 |
| <code>logit0..4</code>                                                                          | out   | 16 signed | Kết quả phân loại cuối                    |

### 0.2 Tín hiệu nội bộ chính

| Tín hiệu             | Width | Mô tả                                                     |
|----------------------|-------|-----------------------------------------------------------|
| <code>state</code>   | 8     | Trạng thái FSM hiện tại                                   |
| <code>substep</code> | 3     | Sub-step bên trong một state (cho pipeline read-wait-use) |
| <code>t_cnt</code>   | 10    | Vòng lặp theo trục thời gian                              |
| <code>c_grp</code>   | 3     | Vòng lặp theo nhóm output channel (mỗi nhóm = 16 channel) |
| <code>c_grp_m</code> | 4     | Vòng lặp theo nhóm Mamba inner channel                    |

| Tín hiệu             | Width     | Mô tả                                                              |
|----------------------|-----------|--------------------------------------------------------------------|
| mac_idx              | 8         | Index input channel trong MAC reduction                            |
| k_idx                | 6         | Index kernel position (cho conv k=9/19/39)                         |
| branch_id            | 3         | 0=Bot, 1=B1, 2=B2, 3=B3, 4=B4                                      |
| s_idx                | 4         | Index state (0..15) trong SSM scan                                 |
| c_grp_br             | 1         | Output group con trong nhánh Inception (cho block 4)               |
| m_rd_addr, m_wr_addr | 15        | Địa chỉ đọc/ghi Memory_System (15-bit)                             |
| w_rd_addr            | 15        | Địa chỉ đọc Weight RAM                                             |
| c_rd_addr            | 15        | Địa chỉ đọc Const Storage                                          |
| m_rd_data, m_wr_data | 256       | Data line, 16 lane × 16-bit                                        |
| w_rd_data            | 256       | Weight line, 16 lane × 16-bit                                      |
| c_rd_data            | 256       | Const line, 16 lane × 16-bit                                       |
| pe_A                 | 16 signed | Scalar broadcast vào tất cả 16 PE (chế độ MAC)                     |
| pe_A_vec             | 256       | Vector input cho 16 PE (chế độ element-wise)                       |
| pe_B                 | 256       | Vector input thứ hai cho 16 PE                                     |
| pe_a_is_vector       | 1         | Chọn <b>pe_A</b> (0, broadcast) hay <b>pe_A_vec</b> (1, lane-wise) |
| pe_op_mode           | 2         | MODE_MAC / MODE_MUL / MODE_ADD                                     |
| pe_clear             | 1         | Reset accumulator trước MAC chuỗi mới                              |
| pe_out               | 256       | Output 16 lane × 16-bit từ PE Array                                |
| bank_sel             | 1         | Định tuyến read/write giữa ram_a và ram_b                          |
| m_we                 | 1         | Write enable cho Memory_System                                     |
| incep_reg            | 256       | Buffer giữ inception output cho FIN                                |
| max_buf              | 256       | Accumulator max-pool                                               |
| h_reg                | 256       | SSM hidden state                                                   |
| norm_sq_acc          | 40 signed | Accumulator sum-of-squares cho RMSNorm                             |
| norm_S_reg           | 16 signed | Latched RSqrt ROM output                                           |
| gap_sum[0:7][0:15]   | 24 signed | Accumulator GAP per (c_grp, lane)                                  |
| gap_q_reg[0:7]       | 256       | GAP output sau finalize                                            |
| fc_acc               | 40 signed | Accumulator FC scalar                                              |
| fc_gap_word          | 256       | Cached gap word cho FC inner loop                                  |

### 0.3 Định dạng Q4.11

- Mỗi giá trị: 16-bit signed two's complement
- 4 bit integer, 11 bit phân số
- Khoảng float biểu diễn:  $[-16.0, +16.0]$
- Độ phân giải:  $1/2048 \approx 0.000488$
- Float  $f \rightarrow$  int16  $q$ :  $q = \max(-32768, \min(32767, \text{floor}(f \times 2048)))$
- Int16  $q \rightarrow$  float  $f$ :  $f = q / 2048.0$

## 0.4 Một số constants quan trọng

```

FRAC_BITS      = 11
SCALE          = 2048 = 1 << 11
INV_T_Q15      = 131  (= round(2^15 / 250), dùng cho GAP)
N_RMS_PREC     = 6    (RMSNorm v2 extra precision bits)
K_RMS_ROM      = 23170  $\approx \sqrt{2^7} \times 2048$ 
B_ENC_IN_BASE  = 19000 (ram_b)
W_ENC_BASE     = 14000 (ram_weight)
W_FC_BASE      = 14064 (ram_weight)
C_ENC_BIAS     = 64    (ram_const)
C_FC_BIAS      = 68    (ram_const)
A_INPUT_BASE   = 0     (ram_a, encoder out / block 0 input)
B_P1_OUT       = 0     (ram_b, P1 output)

```

## 1. Khoảnh khắc đầu: vector arrive

### 1.1 Bên ngoài FPGA (host CPU)

Mẫu ECG đến từ tập kiểm tra PTB-XL ở dạng tensor PyTorch  $(1, 1000, 12)$  — 1 sample  $\times$  1000 timestep  $\times$  12 lead. Mỗi giá trị là float32 đã chuẩn hóa (z-score trung bình 0, std 1, phần lớn nằm trong  $[-5, +5]$ ).

Trong forward path của PyTorch:

```

def forward(self, x):
    x = x.transpose(-1, -2)
    x = self.encoder(x)
    x = self.layers(x)
    x = x.mean(dim=-1)
    x = self.classifier(x)
    return x
# x: (B, T=1000, C_in=12)
# → (B, 12, 1000)
# Conv1d(12→64, k=1) + BN
# 5 ITMBlock + 2 MaxPool
# GAP → (B, 128)
# Linear(128, 5)

```

Host chuẩn bị quantize  $x$  sang Q4.11 trước khi DMA. Cụ thể  $\text{wav\_q}[ch, t] = \text{clip}(\text{floor}(x[t, ch] \times 2048), -32768, +32767)$  cho mỗi  $ch \in [0, 12)$  và mỗi  $t \in [0, 1000)$ . Đây là 12000 giá trị int16.

### 1.2 Pack thành 256-bit word cho DMA

Mỗi 256-bit DMA word chứa 16 lane  $\times$  16-bit. Encoder cần input "1 word per timestep, 12 lane valid + 4 lane zero-pad". Host pack:

```

for t in 0..999:
    word_t = 0
    for ch in 0..11:
        word_t[ch*16 +: 16] = wav_q[ch, t]    # 12 lane đầu
    # lane 12, 13, 14, 15 giữ 0
    DMA_write(target=1, addr=B_ENC_IN_BASE + t, wdata=word_t)

```

Tổng cộng 1000 DMA write transactions sẽ đẩy raw waveform vào `ram_b` ở vùng [19000, 19999].

### 1.3 Tín hiệu DMA write trong 1 transaction

Ví dụ host muốn ghi waveform của timestep `t=42` vào địa chỉ 19042 trên `ram_b`:

```

Cycle N (negedge):
    dma_write_en = 1
    dma_target   = 2'b01    // ram_b
    dma_addr     = 15'd19042
    dma_wdata    = {128'h0, wav_q[11,42], wav_q[10,42], ..., wav_q[0,42]}
                  (lane 0 ở LSB của 256-bit)

Cycle N (posedge):
    Memory_System routing logic detect target=1 + write_en:
        we_b      = 1
        addr_a_wr = dma_addr[14:0]    // 14-bit + we_b decide
        din_b     = dma_wdata
    BRAM_256b ram_b internal:
        always @(posedge clk) if (we_a) ram[addr_a] <= din_a;
        → ram_b.ram[19042] <= dma_wdata

Cycle N+1 (negedge):
    dma_write_en = 0
    (transaction xong)

```

Quá trình lặp 1000 lần (negedge–posedge–negedge), tổng 2000 clock edge chỉ cho việc load 1 sample waveform. Ở 100 MHz:  $1000 \times 10 \text{ ns} = 10 \mu\text{s}$ .

### 1.4 Các DMA load khác (boot-time, một lần)

Trước khi xử lý sample, host cũng DMA-load các weights cố định:

- **Encoder weight** ( $64 \times 12 \rightarrow 64$  word `ram_weight` ở [14000, 14063])
- **Encoder bias** (64 channel  $\rightarrow 4$  word `ram_const` ở [64, 67])
- **FC weight** ( $5 \times 128 \rightarrow 40$  word `ram_weight` ở [14064, 14103])
- **FC bias** (5 channel  $\rightarrow 1$  word `ram_const` ở [68])

Mỗi block 0..4 cũng load weight riêng vào `ram_weight` ở [0, ~13000) trước khi block đó chạy, nhưng encoder/FC weight ở dải [14000+) không bị overwrite vì các block chỉ dùng tới ~13000.

## 2. Phase Encoder — chuyển 12 lead → 64 channel

### 2.1 PyTorch tương đương

```
self.encoder = nn.Sequential(
    nn.Conv1d(12, 64, kernel_size=1), # weight shape (64, 12, 1)
    nn.BatchNorm1d(64),               # weight/bias shape (64,)
)
# x: (1, 12, 1000) → conv → (1, 64, 1000) → BN → (1, 64, 1000)
```

Conv1d với k=1 thực chất là phép linear projection per-timestep:

$$\text{out}[c, t] = \sum_{j=0..11} W_{\text{conv}}[c, j, 0] \cdot x[j, t] + b_{\text{conv}}[c]$$

Sau khi BN, công thức gộp lại (fuse\_conv\_bn) thành:

$$\begin{aligned} W_{\text{fused}}[c, j] &= W_{\text{conv}}[c, j, 0] \cdot \gamma[c] / \sqrt{\text{var}[c] + \text{eps}} \\ b_{\text{fused}}[c] &= (b_{\text{conv}}[c] - \mu[c]) \cdot \gamma[c] / \sqrt{\text{var}[c] + \text{eps}} + \beta[c] \\ \text{out}[c, t] &= \sum W_{\text{fused}}[c, j] \cdot x[j, t] + b_{\text{fused}}[c] \end{aligned}$$

Trong FPGA, mọi giá trị đã quantize Q4.11. Phép MAC trở thành integer:

$$\begin{aligned} \text{sum\_q}[c, t] &= \sum_{j=0..11} W_{\text{q}}[c, j] \cdot X_{\text{q}}[j, t] && // \text{ 40-bit accumulator} \\ \text{enc\_q}[c, t] &= \text{sat\_add16}( \text{sat16}( \text{sum\_q}[c, t] \gg 11 ), b_{\text{q}}[c] ) \end{aligned}$$

### 2.2 Memory layout cho Encoder

**Input ram\_b[19000..19999]:** 1 word/timestep, lane 0..11 = X[lead 0..11, t], lane 12..15 = 0.

**Weight ram\_weight[14000..14063]:** 64 entries, tổ chức theo  $(c_{\text{grp\_out}} \times 16$

- in\_ch). Cụ thể tại địa chỉ  $W_{\text{ENC\_BASE}} + c_{\text{grp\_out}} \times 16 + \text{in\_ch}$ :
- Lane  $i$  ( $i \in 0..15$ ) chứa  $W_{\text{fused}}[c_{\text{grp\_out}} \times 16 + i, \text{in\_ch}]$
- Tức là cùng 1 input channel in\_ch, 16 output channel khác nhau, packed trong 1 word

Bố cục này khớp với cách MAC: với  $\text{pe\_A}$  = scalar input channel in\_ch, và  $\text{pe\_B}$  = 16-lane weight cho 16 output channel, mỗi PE accumulate cho output channel của lane đó.

**Bias ram\_const[64..67]:** 4 word, lane  $i$  của word  $c_{\text{grp\_out}} = b_{\text{fused}}[c_{\text{grp\_out}} \times 16 + i]$ .

**Output ram\_a[0..3999]:** encoder ghi vào  $A_{\text{INPUT\_BASE}} + t \times 4 + c_{\text{grp\_out}}$ . 4 word/timestep, mỗi word 16 output channel. Đây cũng là layout mà Block 0 P1 sẽ đọc làm input — encoder output thẳng thay cho input mà host phải DMA.

## 2.3 FSM Encoder — overview

5 state, lặp 4000 lần:

```
S_ENC_MAC (12 substep MAC) → S_ENC_WAIT → S_ENC_WRITE → S_ENC_NEXT
                                     |
                                     └─ next c_grp_out
                                     └─ hoặc next t
```

Tổng cycle:  $4 \text{ c\_grp} \times 1000 \text{ t} \times 16 \text{ cycle/iteration} \approx 64,000 \text{ cycle} \approx 640 \mu\text{s}$ .

## 2.4 S\_IDLE → S\_ENC\_MAC: kickoff

Host pulse `start = 1` trong khi `enc_mode = 1`, `head_mode = 0`, `T_ENC = 10'd1000`, các config block 0 (`CH_IN=4`, `CH_OUT=4`, ...). Trong cycle này:

```
Cycle 0 (posedge): state = S_IDLE
  if (start) {
    // Clear done flags
    done_phase1 ← 0; done_inception ← 0; done_mamba ← 0
    done_all ← 0; done_encoder ← 0; done_gap ← 0; done_fc ← 0
    // Reset counters
    t_cnt ← 0; c_grp ← 0; c_grp_m ← 0; mac_idx ← 0
    substep ← 0; branch_id ← 0; s_idx ← 0; c_grp_br ← 0
    // Latch mode flags
    enc_mode_reg ← 1; head_mode_reg ← 0
    cascade_mode_reg ← 1; need_pool_reg ← 0
    // Reset GAP/FC state (vì chuẩn bị cho cả pipeline)
    gap_c_grp ← 0; gap_t ← 0; ...
    // Branch: vì enc_mode = 1
    bank_sel ← 1           // sẽ read ram_b, write ram_a
    c_rd_addr ← C_ENC_BIAS // = 64
    state ← S_ENC_MAC
  }
```

Cuối cycle 0: state đã được lệnh thành S\_ENC\_MAC, `bank_sel = 1`, `c_rd_addr = 64`. Vì `c_rd_addr` đã thay đổi, `Const_Storage` sẽ đọc `ram_const[64]` và đến cuối cycle 1 `c_rd_data` sẽ valid (BRAM read 1-cycle latency).

## 2.5 Vòng lặp Encoder: chi tiết (`t = 0`, `c_grp_out = 0`)

Iteration (`t=0`, `c_grp_out=0`) tính 16 output channel: `enc[0..15, 0]`.

### Cycle 1: S\_ENC\_MAC, substep = 0, mac\_idx = 0

```
Hardware:
m_rd_addr ← B_ENC_IN_BASE + t_cnt = 19000 + 0 = 19000
w_rd_addr ← W_ENC_BASE + c_grp × 16 + mac_idx = 14000 + 0 + 0 = 14000
```

```

pe_A      ← 16'sd0      // chưa dùng, sẽ overwrite ở substep 2
substep   ← 1

```

Cuối cycle 1:

- ram\_b internal: addr\_b = 19000 đang được đẩy vào port → đầu cycle 2, m\_rd\_data sẽ là word ram\_b[19000] = waveform timestep 0
- ram\_weight internal: tương tự, đầu cycle 2 w\_rd\_data sẽ là encoder weight row 0 (in\_ch=0)
- c\_rd\_data: từ cycle trước đã có ram\_const[64] = encoder bias word 0 (output channel 0..15)

### Cycle 2: S\_ENC\_MAC, substep = 1, mac\_idx = 0

Hardware:

```

m_rd_data đã valid: = X_q[lane 0..11, t=0]  packed như sau
  m_rd_data[ 0 +: 16] = X_q[0, 0]
  m_rd_data[16 +: 16] = X_q[1, 0]
  ...
  m_rd_data[176 +: 16] = X_q[11, 0]
  m_rd_data[192..255] = 0 (4 lane pad)
w_rd_data đã valid: 16 lane = W_fused[0..15, in_ch=0]
  w_rd_data[i*16 +: 16] = W_fused_q[i, 0]

pe_A      ← 16'sd0      // vẫn 0
substep    ← 2

```

Trong cycle này, FSM chỉ wait 1 cycle để chắc chắn read data đã ổn định trước khi feed vào PE Array.

### Cycle 3: S\_ENC\_MAC, substep = 2, mac\_idx = 0

Hardware:

```

pe_A      ← m_rd_data[mac_idx[3:0] × 16 +: 16] = m_rd_data[0 +: 16] = X_q[0,
0]
pe_B      ← w_rd_data      // full 256-bit, 16 lane
pe_clear  ← (mac_idx == 0) // = 1 → reset accumulator của 16 PE
// mac_idx != mac_target (= 11), nên next:
mac_idx    ← 1
substep    ← 0

```

Tại posedge sắp tới, PE Array sẽ:

- Mỗi PE lane *i*: lấy `pe_a_lane = (a_is_vector ? in_A_vec[i] : in_A) = pe_A = X_q[0, 0]` (broadcast)
- Mỗi PE lane *i*: lấy `pe_b_lane = pe_B[i*16 +: 16] = W_fused_q[i, 0]`
- Mỗi PE lane *i*: `mult_i = X_q[0,0] × W_fused_q[i, 0]` (32-bit)
- Vì `pe_clear = 1`: `acc_raw_i ← mult_ext_i` (reset accumulator)
- `out_val_i ← sat16(acc_raw_i >>> 11)` (đăng ký vào FF của PE)

→ 16 PE bắt đầu accumulator chain mới với product đầu tiên.

#### Cycle 4: S\_ENC\_MAC, substep = 0, mac\_idx = 1

Hardware:

```
m_rd_addr ← 19000 + 0 = 19000    // không đổi, vì t=0 vẫn vậy
w_rd_addr ← 14000 + 0 + 1 = 14001 // weight row 1, in_ch = 1
pe_A      ← 0
substep   ← 1
```

#### Cycle 5: substep = 1, mac\_idx = 1

```
m_rd_data đã ổn định ở X_q[0..11, 0] từ trước
w_rd_data mới valid = W_fused_q[0..15, in_ch=1]
substep ← 2
```

#### Cycle 6: substep = 2, mac\_idx = 1

```
pe_A      ← m_rd_data[1*16 +: 16] = X_q[1, 0]
pe_B      ← w_rd_data
pe_clear  ← 0    // không reset, tiếp tục accumulate
mac_idx   ← 2
substep   ← 0
```

Tại posedge:

- PE lane  $i$ :  $\text{mult}_i = X_q[1, 0] \times W_{\text{fused\_q}}[i, 1]$
- $\text{acc\_raw}_i \leftarrow \text{acc\_raw}_i + \text{mult\_ext}_i$

Sau 12 lần lặp như vậy ( $\text{mac\_idx} = 0..11$ ), accumulator của PE lane  $i$  chứa  $\sum_{j=0..11} X_q[j, 0] \times W_{\text{fused\_q}}[i, j]$  = sum theo công thức encoder.

#### Cycle 36: S\_ENC\_MAC, substep = 2, mac\_idx = 11 (lần cuối)

```
pe_A      ← X_q[11, 0]
pe_B      ← w_rd_data
pe_clear  ← 0
// mac_idx == 11 (= max), nên:
state ← S_ENC_WAIT
mac_idx & substep giữ nguyên cho cycle này
```

#### Cycle 37: S\_ENC\_WAIT

```
state ← S_ENC_WRITE
```

Wait 1 cycle để PE Array commit kết quả vào **pe\_out** (Unified\_PE có registered output).

### Cycle 38: S\_ENC\_WRITE

```
m_we      ← 1
bank_sel  ← 1    // route write to ram_a
m_wr_addr ← A_INPUT_BASE + t_cnt × 4 + c_grp = 0 + 0 + 0 = 0
m_wr_data: each lane i ←
    sat_add16(pe_out[i*16 +: 16], c_rd_data[i*16 +: 16])
    // pe_out[i*16 +: 16] = sat16(accumulator_i >>> 11) ≈ MAC result for output
    channel i
    // c_rd_data[i*16 +: 16] = b_fused_q[i] (bias for output channel i)
state ← S_ENC_NEXT
```

Tại posedge:

- $\text{ram\_a.we\_a} = 1, \text{addr\_a} = 0, \text{din\_a} = \text{m\_wr\_data} \rightarrow \text{ram\_a}[0] \leftarrow \text{m\_wr\_data}$
- $\text{m\_wr\_data lane } i = \text{enc\_q}[i, 0]$  (encoder output cho output channel  $i$ , timestep 0)

→ Sau cycle 38,  $\text{ram\_a}[0]$  chứa encoder output 16 channel đầu (output ch 0..15) cho timestep 0.

### Cycle 39: S\_ENC\_NEXT

```
mac_idx ← 0
substep ← 0
m_we     ← 0    // (cũng đã set 0 mặc định ở đầu always block)

// c_grp (= 0) != 3, nên:
c_grp    ← 1
c_rd_addr ← C_ENC_BIAS + c_grp + 1 = 64 + 0 + 1 = 65    // next bias word
state    ← S_ENC_MAC
```

Lưu ý: **c\_rd\_addr** được set sớm để có thời gian Const\_Storage đọc xong word mới trước khi cần ở cycle S\_ENC\_WRITE tiếp theo.

## 2.6 Lặp lại cho $c\_grp\_out = 1..3$ ( $t = 0$ )

Tương tự như  $c\_grp\_out = 0$ :

- ( $t=0, c\_grp\_out=1$ ): tính output channel 16..31, ghi vào  $\text{ram\_a}[1]$
- ( $t=0, c\_grp\_out=2$ ): tính output channel 32..47, ghi vào  $\text{ram\_a}[2]$
- ( $t=0, c\_grp\_out=3$ ): tính output channel 48..63, ghi vào  $\text{ram\_a}[3]$

Cycle count cho một ( $t, c\_grp$ ):

- substep 0/1/2 cho mac\_idx = 0..11:  $12 \times 3 = 36$  cycle
- S\_ENC\_WAIT: 1 cycle
- S\_ENC\_WRITE: 1 cycle
- S\_ENC\_NEXT: 1 cycle (mặc dù không strictly đợi, FSM vẫn tiêu cycle này)

Total: 39 cycle/iteration  $\approx 16$  (làm tròn nhưng thực ra 39 với detailed)

## 2.7 Chuyển timestep: $t = 0 \rightarrow t = 1$

Khi đã hoàn thành ( $t=0$ ,  $c\_grp=3$ ), tại cycle S\_ENC\_NEXT:

```
mac_idx ← 0
substep ← 0
m_we ← 0

// c_grp == 3, nên:
c_grp ← 0
// t_cnt != T_ENC - 1, nên:
t_cnt ← 1 // hoặc t_cnt + 1
c_rd_addr ← C_ENC_BIAS // = 64, reset cho c_grp = 0
state ← S_ENC_MAC
```

Iteration ( $t=1$ ,  $c\_grp\_out=0$ ) lặp y hệt nhưng  $m\_rd\_addr = 19001$  (đọc waveform timestep 1),  $m\_wr\_addr = A\_INPUT\_BASE + 1 \times 4 + 0 = 4$ .

## 2.8 Kết thúc Encoder: $t = 999$ , $c\_grp = 3$

Sau khi ghi xong word cuối cùng tại  $ram\_a[3999]$  ( $= A\_INPUT\_BASE + 999 \times 4 + 3$ ), FSM rơi vào S\_ENC\_NEXT lần cuối:

```
mac_idx ← 0; substep ← 0; m_we ← 0
// c_grp == 3 (cuối)
c_grp ← 0
// t_cnt == T_ENC - 1 == 999, nên: encoder done
done_encoder ← 1 // pulse 1 cycle
enc_phase ← 0
t_cnt ← 0
t_cnt_zero // reset registered strides t_stride_in/m/out/xp về 0
bank_sel ← 0 // back to P1 mode: read ram_a, write ram_b
c_rd_addr ← C_P1_BIAS // = 0
need_pool_reg ← need_pool (re-latch, vẫn 0 cho block 0)
cascade_mode_reg ← cascade_mode (vẫn 1)
t_out_cnt ← 0
state ← S_P1_MAC
```

→ Bắt đầu Block 0 P1 ngay lập tức, KHÔNG cần host pulse start lại.

Sau encoder:

- `ram_a[0..3999]` chứa encoder output: `enc_q[0..63, 0..999]` packed 16 channel/word, 4 word/timestep, 1000 timestep
- Tất cả memory khác chưa được ghi (block 0 weight đã có sẵn từ DMA boot)

### 3. Phase 1 — Block 0 P1 (Conv1D 1×1 + BN, 64 → 64)

#### 3.1 PyTorch tương đương

```
class ITMBlock:
    def __init__(...):
        self.conv = nn.Sequential(
            nn.Conv1d(64, 64, kernel_size=1),
            nn.BatchNorm1d(64),
        )
    # P1 = self.conv(x), với x là encoder output (1, 64, 1000)
```

Sau fuse\_conv\_bn, công thức P1:

$$p1[c, t] = \sum_{j=0..63} W_{p1}[c, j] \times enc[j, t] + b_{p1}[c]$$

Kích thước MAC: 64 input channel × 64 output channel, T = 1000 timestep.

#### 3.2 Memory layout

**Input `ram_a[0..3999]`:** từ encoder. 4 word/timestep, mỗi word 16 channel.

- `ram_a[t × 4 + c_grp_in]` với `c_grp_in` ∈ [0, 3]
- Lane `i` của word này = `enc_q[c_grp_in × 16 + i, t]`

**Weight `ram_weight[0..255]`:** P1 cho block 0.

- `W_P1_BASE` = 0
- Tại địa chỉ `W_P1_BASE + c_grp_out × d_in + mac_idx` (`d_in` = 64, `c_grp_out` ∈ [0,4), `mac_idx` ∈ [0, 64))
- Lane `i` chứa `W_p1_q[c_grp_out × 16 + i, mac_idx]`

**Bias `ram_const[0..3]`:** 4 word, lane `i` của word `c_grp_out` = `b_p1_q[c_grp_out × 16 + i]`.

**Output `ram_b[B_P1_OUT + t × 4 + c_grp_out]`:** 4 word/timestep, mỗi word 16 output channel.

#### 3.3 FSM P1 — overview

```
S_P1_MAC (3 substep, mac_idx 0..63) → S_P1_WAIT → S_P1_WRITE → S_P1_NEXT
```

Cycle count:  $4 \text{ c\_grp} \times 1000 \text{ t} \times (3 \times 64 + 3) = 4 \times 1000 \times 195 \approx 780,000 \text{ cycle}$ .

### 3.4 Vòng lặp P1: detailed (t = 0, c\_grp\_out = 0)

#### Cycle 0 (kế thừa từ S\_ENC\_NEXT): S\_P1\_MAC, substep = 0, mac\_idx = 0

```

m_rd_addr ← A_INPUT_BASE + t_stride_in + mac_idx[7:4]
           = 0 + 0 + 0 = 0    // word đầu tiên của t=0
           // mac_idx[7:4] = 0 vì mac_idx = 0
w_rd_addr ← W_P1_BASE + c_grp × d_in + mac_idx
           = 0 + 0 × 64 + 0 = 0
pe_A      ← 0
substep   ← 1

```

Lưu ý: P1 dùng `t_stride_in` (registered accumulator). Tại đầu phase này `t_stride_in = 0` (đã reset bởi `t_cnt_zero` khi vào). Stride increment mỗi khi t tăng: `t_stride_in += CH_IN = 4`.

#### Cycle 1: substep = 1, mac\_idx = 0

```

m_rd_data valid: ram_a[0] = encoder out cho (ch 0..15, t=0)
w_rd_data valid: ram_weight[0] = W_p1_q[0..15, 0]    // 16 output ch, input ch 0
pe_A          ← 0
substep       ← 2

```

#### Cycle 2: substep = 2, mac\_idx = 0

```

pe_A      ← m_rd_data[mac_idx[3:0] × 16 +: 16] = m_rd_data[0 +: 16] = enc_q[0, 0]
pe_B      ← w_rd_data
pe_clear  ← (mac_idx == 0) = 1    // reset accumulator
mac_idx   ← 1
substep   ← 0

```

PE Array tại posedge:

- 16 PE: `mult_i = enc_q[0, 0] × W_p1_q[i, 0]`
- Reset accumulator: `acc_raw_i ← mult_ext_i`

#### Cycle 3: substep = 0, mac\_idx = 1

```

m_rd_addr ← A_INPUT_BASE + t_stride_in + mac_idx[7:4]
           = 0 + 0 + 0 = 0    // mac_idx = 1, [7:4] = 0 → cùng word
w_rd_addr ← 0 + 0 + 1 = 1
substep   ← 1

```

#### Cycle 4: substep = 1, mac\_idx = 1

```

w_rd_data valid: ram_weight[1] = W_p1_q[0..15, 1]
m_rd_data: vẫn ram_a[0]
substep ← 2

```

#### Cycle 5: substep = 2, mac\_idx = 1

```

pe_A      ← m_rd_data[1 × 16 +: 16] = enc_q[1, 0]
pe_B      ← w_rd_data
pe_clear  ← 0
mac_idx   ← 2

```

... (lặp 15 lần với mac\_idx[7:4] = 0, cùng word ram\_a[0])

#### Cycle 47: substep = 2, mac\_idx = 15

```

pe_A      ← enc_q[15, 0]
pe_B      ← w_rd_data = W_p1_q[0..15, 15]
pe_clear  ← 0
mac_idx   ← 16

```

#### Cycle 48: substep = 0, mac\_idx = 16

```

m_rd_addr ← A_INPUT_BASE + t_stride_in + mac_idx[7:4]
           = 0 + 0 + 1 = 1    // mac_idx = 16, [7:4] = 1 → word kể ram_a[1]
w_rd_addr ← 0 + 0 + 16 = 16
substep   ← 1

```

#### Cycle 49: substep = 1, mac\_idx = 16

```

m_rd_data NEW valid: ram_a[1] = encoder out cho (ch 16..31, t=0)
w_rd_data NEW valid: W_p1_q[0..15, 16]
substep ← 2

```

#### Cycle 50: substep = 2, mac\_idx = 16

```

pe_A      ← m_rd_data[mac_idx[3:0] × 16 +: 16] = m_rd_data[0 +: 16] = enc_q[16, 0]
pe_B      ← w_rd_data

```

```
pe_clear ← 0
mac_idx ← 17
```

... (tiếp tục đến mac\_idx = 63)

### Cycle 191: substep = 2, mac\_idx = 63 (lần cuối)

```
pe_A      ← m_rd_data[15 × 16 +: 16] = enc_q[63, 0]
pe_B      ← w_rd_data = W_p1_q[0..15, 63]
pe_clear ← 0
// mac_idx == d_in_last == 63
state ← S_P1_WAIT
```

### Cycle 192: S\_P1\_WAIT

```
state ← S_P1_WRITE
```

### Cycle 193: S\_P1\_WRITE

```
m_we      ← 1
m_wr_addr ← B_P1_OUT + t_stride_out + c_grp = 0 + 0 + 0 = 0
m_wr_data: each lane i ←
    sat_add16(pe_out[i*16 +: 16], c_rd_data[i*16 +: 16])
state ← S_P1_NEXT
```

bank\_sel hiện = 0, nên write routes to ram\_b. ram\_b[0] ← P1 output cho (ch 0..15, t=0).

### Cycle 194: S\_P1\_NEXT

```
mac_idx ← 0
substep ← 0
// c_grp == 0, ch_out_last[2:0] == 3, nên:
c_grp    ← 1
c_rd_addr ← C_P1_BIAS + c_grp + 1 = 0 + 0 + 1 = 1    // next bias word
state    ← S_P1_MAC
```

## 3.5 Tổng kết cycle cho 1 (t, c\_grp) iteration

| Phase                            | Cycle |
|----------------------------------|-------|
| 64 × MAC substeps (3 cycle each) | 192   |

| Phase        | Cycle      |
|--------------|------------|
| S_P1_WAIT    | 1          |
| S_P1_WRITE   | 1          |
| S_P1_NEXT    | 1          |
| <b>Total</b> | <b>195</b> |

Per block 0 P1:  $4 \text{ c\_grp} \times 1000 \text{ t} \times 195 = 780,000 \text{ cycle}$ .

### 3.6 Chuyển timestep: kích hoạt t\_cnt\_inc

Khi c\_grp đã hoàn thành 0..3 cho timestep t hiện tại, S\_P1\_NEXT:

```

mac_idx ← 0; substep ← 0
// c_grp == 3 (cuối)
c_grp ← 0
// t_cnt != t_last (= 999)
t_cnt ← t_cnt + 1
t_cnt_inc; // task: increment all 4 registered strides
  t_stride_in ← t_stride_in + CH_IN // += 4
  t_stride_m ← t_stride_m + CH_M // += 8
  t_stride_out ← t_stride_out + CH_OUT // += 4
  t_stride_xp ← t_stride_xp + 3 // += 3 (cho M4)
c_rd_addr ← C_P1_BIAS // reset cho c_grp=0
state ← S_P1_MAC

```

→ Stride register update đồng bộ với t\_cnt, không cần combinational multiplier trên data path (đây là CP-4 fix tăng Fmax).

### 3.7 Kết thúc P1: chuyển sang Inception

Sau khi P1 xong với t = 999, c\_grp = 3, S\_P1\_NEXT:

```

mac_idx ← 0; substep ← 0
// c_grp == 3 (cuối)
c_grp ← 0
// t_cnt == t_last == 999
done_phase1 ← 1 // pulse
t_cnt ← 0; t_cnt_zero // reset strides
k_idx ← 0
branch_id ← 0 // start với nhánh Bot
bank_sel ← 1 // (Inception BR read ram_b P1_out, write ram_a BOT_OUT)
c_grp_br ← 0 // start với group output đầu (cho block 4)
state ← S_BR_MAC

```

Sau Phase 1:

- ram\_b[0..3999] chứa P1\_out: p1\_q[c, t] cho c ∈ [0, 63], t ∈ [0, 999]
- ram\_a[0..3999] vẫn chứa encoder out (sẽ bị overwrite ở các phase sau khi cần)
- done\_phase1 đã = 1

## 4. Phase 2 — Inception (Block 0)

### 4.1 PyTorch tương đương

```
class BaseInceptionBlock:
    def __init__(self, d_model=64):
        dim = d_model // 4 # = 16 cho block 0
        self.bottleneck = nn.Conv1d(d_model, dim, kernel_size=1, bias=False)
        self.conv4 = nn.Conv1d(dim, dim, kernel_size=39, padding=19, bias=False)
        self.conv3 = nn.Conv1d(dim, dim, kernel_size=19, padding=9, bias=False)
        self.conv2 = nn.Conv1d(dim, dim, kernel_size=9, padding=4, bias=False)
        self.maxpool = nn.MaxPool1d(kernel_size=3, stride=1, padding=1)
        self.conv1 = nn.Conv1d(d_model, dim, kernel_size=1, bias=False)
        self.bn = nn.BatchNorm1d(d_model)
        self.relu = nn.ReLU()

    def forward(self, x):
        bot = self.bottleneck(x) # (1, 16, 1000)
        out4 = self.conv4(bot) # (1, 16, 1000)
        out3 = self.conv3(bot) # (1, 16, 1000)
        out2 = self.conv2(bot) # (1, 16, 1000)
        out1 = self.conv1(self.maxpool(x)) # (1, 16, 1000) (maxpool input, conv1
k=1)
        cat = torch.cat((out1, out2, out3, out4), dim=1) # (1, 64, 1000)
        return self.relu(self.bn(cat))
```

Tổng cộng 5 nhánh tính song song trong PyTorch, gộp lại thành 64 channel. **Trên FPGA, 5 nhánh tính tuần tự** (chia sẻ PE Array).

### 4.2 Thứ tự nhánh + memory layout

FSM lặp branch\_id = 0..4:

| branch_id | Nhánh | PyTorch op         | Input               | Weight     | Output               | Kernel           |
|-----------|-------|--------------------|---------------------|------------|----------------------|------------------|
| 0         | Bot   | bottleneck(p1)     | B_P1_OUT<br>(ram_b) | W_BOT_BASE | A_BOT_OUT<br>(ram_a) | k=1              |
| 1         | B1    | conv1(maxpool(p1)) | B_P1_OUT            | W_B1_BASE  | A_CH1_OUT            | k=1 +<br>maxpool |
| 2         | B2    | conv2(bot)         | A_BOT_OUT           | W_B2_BASE  | B_CH2_OUT            | k=9,<br>pad=4    |
| 3         | B3    | conv3(bot)         | A_BOT_OUT           | W_B3_BASE  | B_CH3_OUT            | k=19,<br>pad=9   |

| branch_id | Nhánh | PyTorch op              | Input     | Weight    | Output    | Kernel          |
|-----------|-------|-------------------------|-----------|-----------|-----------|-----------------|
| 4         | B4    | <code>conv4(bot)</code> | A_BOT_OUT | W_B4_BASE | B_CH4_OUT | k=39,<br>pad=19 |

Mỗi nhánh sản xuất `dim = 16` channel output. Sau khi cả 5 nhánh xong, FIN phase sẽ concat + BN + ReLU.

### 4.3 Nhánh Bot (branch\_id = 0): chi tiết

Bot là 1×1 conv giống encoder/P1 nhưng giảm channel 64 → 16. Khác P1 ở:

- Input từ B\_P1\_OUT (bank B), output vào A\_BOT\_OUT (bank A)
- Không bias, không BN (per BaselineInceptionBlock spec)
- Chỉ 1 c\_grp\_br loop (vì dim=16 = 1 group cho block 0)

#### State arms trong S\_BR\_MAC (chỉ phần liên quan Bot)

`is_ch64_branch` = (branch\_id == 0 || branch\_id == 1) = true cho Bot/B1. `current_num_in_ch` = d\_out = 64 cho Bot. `current_kernel` = 1. `current_pad` = 0. `is_padding` = false (k=1, pad=0). `mac_target` = 63.

#### Cycle (t=0, c\_grp\_br=0, k\_idx=0, mac\_idx=0): S\_BR\_MAC substep = 0

```

m_rd_addr ← current_data_base + (t_eff × CH_OUT) + mac_idx[7:4]
           = B_P1_OUT + 0 + 0 = 0
w_rd_addr ← current_w_base + br_w_offset + (k_idx × current_num_in_ch) + mac_idx
           = W_BOT_BASE + 0 + 0 + 0
pe_A      ← 0
substep   ← 1

```

#### Cycle next: substep = 1

```

m_rd_data valid: ram_b[0] = p1_q[0..15, 0]    // bank_sel = 1 reads ram_b
w_rd_data valid: ram_weight[W_BOT_BASE] = W_bot_q[0..15, 0]
substep ← 2

```

#### Cycle next: substep = 2

```

pe_A      ← m_rd_data[0+: 16] = p1_q[0, 0]
pe_B      ← w_rd_data
pe_clear  ← (k_idx == 0 && mac_idx == 0) = 1
mac_idx   ← 1
substep   ← 0

```

PE Array: `mult_i = p1_q[0, 0] × W_bot_q[i, 0]`, accumulator reset.

**Repeat 63 lần (mac\_idx = 1..63), tương tự P1 logic**

Sau khi MAC qua tất cả 64 input channel:

```
state ← S_BR_WAIT (substep = 2, mac_idx = 63, k_idx = 0)
```

**S\_BR\_WAIT → S\_BR\_WRITE**

```
m_we      ← 1
m_wr_addr ← current_out_base + (t_cnt × br_dim_groups) + c_grp_br
           = A_BOT_OUT + 0 + 0 = 0
m_wr_data ← pe_out    // 16 lane = bottleneck output cho 16 channel
// bank_sel = 1, write to ram_a (A_BOT_OUT)
state ← S_BR_NEXT
```

Lưu ý: Bot không bias, nên **m\_wr\_data = pe\_out** thẳng, không **sat\_add16**.

**S\_BR\_NEXT**

```
m_we ← 0; mac_idx ← 0; k_idx ← 0; substep ← 0
// k_idx (= 0) == k_target (= 0) cho Bot → kernel xong
// c_grp_br (= 0) == br_grp_last (= 0) cho block 0 → c_grp_br xong
// t_cnt != t_last → next timestep
t_cnt      ← t_cnt + 1; t_cnt_inc
state      ← S_BR_MAC
```

Total cho 1 (t, c\_grp\_br) iteration của Bot:  $\sim 64 \times 3 + 3 = 195$  cycle. Tổng Bot:  $1000 \times 195 \approx 195,000$  cycle.

**4.4 Nhánh B1: MaxPool input + Conv 1×1**

B1 đọc P1\_OUT[t-1], P1\_OUT[t], P1\_OUT[t+1], lấy elementwise max, rồi MAC qua W\_B1.

**S\_BR\_MAC khi branch\_id = 1: substep pattern phức tạp hơn**

```
substep 0: read p1_q[t-1]                (set m_rd_addr cho b1_t_prev)
substep 1: wait
substep 2: latch p1_q[t-1] → max_buf,
           set m_rd_addr cho t_cnt (b1_t)
substep 3: wait
substep 4: latch elem_max16(max_buf, p1_q[t]) → max_buf,
           set m_rd_addr cho b1_t_next
substep 5: wait
substep 6: pe_A ← max_buf[mac_idx[3:0] × 16 +: 16],
```

```
tính elem_max final với m_rd_data,
pe_B ← w_rd_data, MAC
```

Edge case  $t = 0$ :  $b1\_t\_prev = 0$  (giới hạn dưới).  $t = 999$ :  $b1\_t\_next = 999$  (giới hạn trên).

Cycle/iteration: phức tạp hơn Bot, nhưng cùng cấu trúc lặp  $mac\_idx = 0..63$ .

#### 4.5 Nhánh B2/B3/B4: Conv k=9/19/39 trên Bot output

B2 (k=9), B3 (k=19), B4 (k=39) đều conv trên A\_BOT\_OUT (16 input channel, chứ không phải 64). Vì vậy:

- $current\_num\_in\_ch = dim = 16$  cho block 0
- $mac\_idx$  chỉ chạy 0..15 (1 word/timestep)
- Phải lặp qua kernel position  $k\_idx = 0..k-1$
- Padding handling: nếu  $t\_eff = t + k\_idx - pad < 0$  hoặc  $\geq T$ , set  $pe\_A = 0$  (zero padding)

#### Vòng lặp B4 (k=39, branch\_id=4)

Cấu trúc 4 vòng lồng nhau:

```
for c_grp_br in 0..br_grp_last:  # 0 cho block 0 (chỉ 1 group)
  for t in 0..T-1:
    for k_idx in 0..38:
      for mac_idx in 0..15:
        # 3 substep MAC
```

Per timestep:  $39 \times 16 \times 3 + 3 \approx 1875$  cycle. Total B4 block 0:  $1000 \times 1875 = 1.875M$  cycle.

Đây là phép tính **dài nhất trong Inception**. Lý do: B4 có kernel 39 và phải quét toàn bộ  $T=1000$  timestep.

#### 4.6 Tổng kết Phase 2

Cycle count cho Inception block 0:

- Bot: 195K
- B1: ~250K (do extra substeps maxpool)
- B2:  $9 \times 16 \times 3 \times 1000 = 432K$
- B3:  $19 \times 16 \times 3 \times 1000 = 912K$
- B4:  $39 \times 16 \times 3 \times 1000 = 1.872M$
- **Total:** ~3.66M cycle

Memory sau Phase 2:

- A\_BOT\_OUT (ram\_a[0..999]): 16 channel  $\times$  1000 t = 1000 word (overlap với A\_INPUT\_BASE, nhưng input không còn cần)
- A\_CH1\_OUT (ram\_a[16000..16999]): B1 output, 1000 word
- B\_CH2\_OUT (ram\_b[16000..16999]): B2 output, 1000 word
- B\_CH3\_OUT (ram\_b[17000..17999]): B3 output
- B\_CH4\_OUT (ram\_b[18000..18999]): B4 output

Sau cùng, S\_BR\_NEXT cuối:

```
done_inception ← 1    // pulse
t_cnt ← 0; t_cnt_zero
bank_sel ← 1    // chuẩn bị cho NORM (đọc B_P1_OUT)
norm_sq_acc ← 40'd0    // reset RMSNorm accumulator
state ← S_NORM_M1A_SQ_READ
```

→ Chuyển sang Mamba phase, bắt đầu bằng RMSNorm.

## 5. Phase 3 — Mamba (Block 0)

Đây là phase phức tạp nhất, gồm các sub-phase:

1. **NORM\_M1A**: RMSNorm cho M1A
2. **M1A**: in\_proj\_x (linear 64 → 128 = d\_inner) qua RMSNorm output
3. **NORM\_M1B**: RMSNorm lần 2 cho M1B
4. **M1B**: in\_proj\_z (linear 64 → 128)
5. **M2**: depthwise conv1d k=4 + bias trên x\_inner
6. **M3**: SiLU(x\_conv) → u\_safe
7. **M3CP**: copy u\_safe sang B\_U\_SAFE (do bank constraints)
8. **M4**: x\_proj (linear d\_inner → dt\_rank + 2 × d\_state) → 3 output groups
9. **M5**: dt\_proj + bias + softplus → delta\_t
10. **M6A**: tính dA = exp(delta\_t × A) per channel + scalar B
11. **M6B**: SSM scan h\_t = dA × h\_{t-1} + dB × u → y per state
12. **M7**: y\_gated = y × SiLU(z\_gate), tính ở cùng state
13. **M8**: out\_proj (linear d\_inner → d\_in = 64)

### 5.1 NORM\_M1A: RMSNorm per timestep

#### PyTorch

```
class RMSNorm:
    def forward(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + eps) *
self.weight
```

Cho  $x: (B, T, d) = (1, T, 64)$  đối với block 0. Trục -1 là channel dim. Vì input vào Mamba block là  $x.transpose$  để chiều cuối thành d, vì vậy mỗi timestep tính một scalar  $S_t = \text{rsqrt}(\text{mean}(x^2_t))$ , sau đó nhân  $x[:, t, :] \times S_t \times \gamma$ .

#### FPGA — 2 phase

##### Phase 1: sum-of-squares

```

for t in 0..T-1:
  norm_sq_acc ← 0
  for c_grp_in in 0..3: # 4 group × 16 lane = 64 channel
    read p1_q[c_grp_in × 16..+15, t] from ram_b[t × 4 + c_grp_in]
    norm_sq_acc +=  $\sum_{i=0..15} (\text{lane}_i)^2$  # norm_sq16_fn
  # tại đây norm_sq_acc =  $\sum_{c=0..63} p1_q[c, t]^2$ 

  mean_i = norm_sq_acc >> (log2(d) + 2*FB - 1 - N)
           = norm_sq_acc >> (6 + 22 - 1 - 6) = >> 21 # cho d=64
  norm_S_reg ← RSqrt_ROM[clip(mean_i, 0, 8191)]

```

**Phase 2:  $x * y * S_t$**  (đã tính **norm\_S\_reg**):

```

for c_grp_out in 0..7 (= CH_M = 8 cho block 0): # M1A có d_out = d_inner = 128
  for mac_idx in 0..63: # d_in = 64
    read p1_q[mac_idx, t] và y[mac_idx]
    # x_norm = sat16( sat16(p1 × y >> 11) × norm_S_reg >> 11 )
    # rms_norm_out (1 scalar)
    pe_A ← rms_norm_out
    pe_B ← W_M1A_X[c_grp_out × 16..+15, mac_idx]
    MAC vào PE Array (16 output channel song song)
    write x_inner[c_grp_out × 16..+15, t] to A_X_INNER[t × 8 + c_grp_out]

```

### State chi tiết NORM\_M1A\_SQ\_READ → S\_NORM\_M1A\_SQ\_LATCH

|                      |                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------|
| S_NORM_M1A_SQ_READ:  | set m_rd_addr ← B_P1_OUT + t_stride_in + mac_idx[7:4]                                          |
| S_NORM_M1A_SQ_WAIT:  | wait                                                                                           |
| S_NORM_M1A_SQ_LATCH: | norm_sq_acc ← norm_sq_acc + norm_sq16_fn(m_rd_data)<br>(combinational reduce 16 squares + sum) |
| S_NORM_M1A_SQ_NEXT:  | advance mac_idx; nếu xong 4 c_grp_in cho t này →                                               |
| S_NORM_M1A_MEAN      |                                                                                                |
| S_NORM_M1A_MEAN:     | compute mean_i, look up RSqrt_ROM, latch norm_S_reg<br>→ S_M1A_MAC                             |

**norm\_sq16\_fn** là combinational function: nhận 256-bit (16 lane × 16-bit), tính  $16 \times 16\text{-bit}^2 = 16 \times 32\text{-bit}$ , cộng cây tree 16-way thành 40-bit.

Sau khi **norm\_sq\_acc** xong cho timestep **t**, chuyển vào pha 2 (M1A\_MAC).

## 5.2 M1A: x\_inner projection (linear 64 → 128)

**Cycle (t = 0, c\_grp\_out = 0, mac\_idx = 0): S\_M1A\_MAC substep = 0**

```

m_rd_addr ← B_P1_OUT + t_stride_in + mac_idx[7:4]
           = 0 + 0 + 0 = 0

```

```

w_rd_addr ← W_M_X_BASE + c_grp_out × d_out + mac_idx
           = W_M_X_BASE + 0 + 0
c_rd_addr ← C_NORM_W + (mac_idx[7:4])
           = 56 + 0 = 56 (gamma[0..15] cho input channel 0..15)
substep   ← 1

```

**substep = 1: wait**

**substep = 2: compute rms\_norm\_out + MAC**

```

// rms_norm_out tính qua RMSNorm_Mul module:
//   p1_wide = p1 × γ           (1 DSP)
//   p1_reg  = sat16(p1_wide >>> 11) (registered, pipelined)
//   out_wide = p1_reg × norm_S_reg (1 DSP)
//   rms_norm_out = sat16(out_wide >>> 11)
pe_A      ← rms_norm_out
pe_B      ← w_rd_data
pe_clear  ← (mac_idx == 0)
mac_idx   ← 1
substep   ← 0

```

Sau 64 MAC (mac\_idx = 0..63), accumulator của PE lane  $i = \sum_j \text{rms\_norm\_out}[j] \times W_{M1A\_x\_q}[c\_grp\_out \times 16 + i, j]$ .

**S\_M1A\_WRITE**

```

m_we      ← 1
m_wr_addr ← A_X_INNER + t_stride_out + c_grp_out // (t × CH_M + c_grp_out)
// CH_M = 8 cho block 0 → t_stride_out tăng +CH_OUT mỗi t; nhưng cho M1A
// dùng t_stride_m thay (tăng +CH_M)
m_wr_data ← pe_out // không bias cho M1A
state     ← S_M1A_NEXT

```

Lưu ý: A\_X\_INNER overlap với A\_INPUT\_BASE (cùng base 0), nhưng A\_INPUT\_BASE giờ đã DEAD (P1 đã đọc xong, encoder out đã consume). Compact memory map cho phép overlap này.

**M1A NEXT iteration**

$c\_grp\_out$  từ 0..7 cho block 0 ( $CH\_M = 8 \rightarrow 128 / 16 = 8$  group). Lặp 8 lần cho mỗi timestep, sau đó NORM lặp lại cho timestep tiếp theo.

**Tổng cycle M1A 1 timestep:** NORM (~70) + M1A ( $8 \times 195 = 1560$ )  $\approx 1630$  cycle.

**5.3 M1B: z\_gate projection (giống M1A)**

Cùng cấu trúc với M1A nhưng:

- Weight ở W\_M\_Z\_BASE (z\_gate weights)
- Output ghi vào A\_Z\_GATE (ram\_a, [8000..15999])
- Cần RMSNorm lần 2 (vì PE Array đã ghi đè state)

Tổng cycle M1B 1 timestep  $\approx 1630$  cycle.

**Cycle cho M1A + M1B per timestep:**  $\sim 3260$  cycle.

#### 5.4 M2: Depthwise Conv1D k=4 + bias

##### PyTorch

```
self.conv1d = nn.Conv1d(d_inner, d_inner, kernel_size=4, groups=d_inner,
                        padding=3, bias=True)
# x_conv = conv1d(x_inner)
```

Depthwise = mỗi output channel chỉ phụ thuộc 1 input channel cùng index. Pad = 3 (causal pad bên trái).

$$x_{\text{conv}}[c, t] = \sum_{k=0..3} W_{\text{dw}}[c, k] \times x_{\text{inner}}[c, t-3+k] + b_{\text{dw}}[c]$$

(với padding 0 khi  $t-3+k < 0$ )

##### FPGA — Depthwise = element-wise multiply

Vì mỗi channel độc lập (groups=d\_inner), không có MAC reduction qua channels. PE Array dùng chế độ `a_is_vector=1, op_mode=MUL`.

##### Cycle (t, c\_grp\_m, k\_idx=0): S\_M2\_MAC

```
m_rd_addr ← A_X_INNER + m2_t_stride + c_grp_m
           // m2_t_eff = t_cnt + k_idx - 3
w_rd_addr ← W_M_DW_BASE + c_grp_m × 4 + k_idx
pe_A_vec ← m_rd_data    // 16 lane × 16-bit, mỗi lane = x_inner[c_grp_m×16+i, t-3+k_idx]
pe_B      ← w_rd_data    // 16 lane × 16-bit, lane i = W_dw[c_grp_m × 16 + i, k_idx]
pe_a_is_vector ← 1
pe_op_mode ← MODE_MUL
```

PE Array tại posedge: mỗi lane  $i$ : `mult_i = x_inner[c_grp_m×16+i, t-3+k_idx] × W_dw[c_grp_m×16+i, k_idx]`. Accumulator giữ.

Sau 4 lần lặp  $k_{\text{idx}} = 0..3$ , mỗi lane đã accumulate  $\sum x \times W$ . Sau đó:

```
S_M2_WRITE:
    m_we ← 1
```

```

m_wr_addr ← B_X_CONV + t_stride_m + c_grp_m
m_wr_data: each lane ← sat_add16(pe_out[lane], c_rd_data[lane]) // + bias
bank_sel ← 0 // write ram_b

```

bias từ ram\_const ở C\_M\_DW\_BIAS region.

**M2 cycle cost:  $4 \times (3 \text{ substep}) \times 8 \text{ c\_grp\_m} \times T \times 1$ .**

Block 0:  $4 \times 3 \times 8 \times 1000 \approx 96\text{K}$  cycle.

## 5.5 M3: SiLU activation

### PyTorch

```
u = F.silu(x_conv)
```

$\text{SiLU}(x) = x \times \sigma(x)$ ,  $x \in \mathbb{R}$ ,  $x_{\text{conv}}$  là Q4.11.

### FPGA — LUT lookup

```

S_M3_MAC:
    m_rd_addr ← B_X_CONV + t_stride_m + c_grp_m

S_M3_WAIT, S_M3_LATCH:
    silu_in[0..15] ← m_rd_data[0..15 × 16 +: 16] (assigned combinatorially)
    silu_o[0..15] are combinational outputs of Silu_LUT instances

S_M3_WRITE:
    m_we      ← 1
    bank_sel  ← 1 (write ram_a)
    m_wr_addr ← A_X_INNER + t_stride_m + c_grp_m
    m_wr_data ← {silu_o[15], silu_o[14], ..., silu_o[0]}

```

Mỗi Silu\_LUT instance là combinational ROM 256-entry × 16-bit. Index tính:

```

idx = (x_in + 16384) >>> 7 // map [-8, +8) → [0, 256)
silu_out = silu_table[idx] // pre-computed at synth time

```

Wait — đoạn này đang nhằm input width. Để chính xác:  $\text{LUT\_LO} = -(1 \ll (\text{FB}+3)) = -16384$  cho  $\text{FB}=11$ .  
Then  $\text{LUT\_SHIFT} = \text{FB} - 4 = 7$ . Index =  $\text{`(x\_in - LUT\_LO)}`$

$\text{LUT\_SHIFT} = (x + 16384) \gg 7 \in [0, 256)$ .

Out of range:  $x < -8 \rightarrow 0$ ,  $x \geq +8 \rightarrow x$  ( $\text{silu}(x) \rightarrow x$  for large  $x$ ).

## 5.6 M3CP: copy A\_X\_INNER → B\_U\_SAFE

PyTorch không có thao tác này — đây là **artifact của memory layout FPGA**. Lý do: M5 sẽ ghi delta vào A\_X\_INNER, đề u; nhưng M6A, M6B sau đó vẫn cần u để tính  $dB \times u$  và  $D \times u$ . Vì vậy phải copy u ra một region khác (B\_U\_SAFE) trước khi M5 chạy.

```
S_M3CP_READ: m_rd_addr ← A_X_INNER + ...
S_M3CP_WAIT: wait
S_M3CP_LATCH: m_wr_data ← m_rd_data
S_M3CP_WRITE: bank_sel ← 0, m_wr_addr ← B_U_SAFE + ..., m_we ← 1
```

Pure copy, no compute. Cost:  $\sim 5 \times CH\_M \times T = 5 \times 8 \times 1000 = 40K$  cycle cho block 0.

(Note: RU-1 optimization plan đề xuất loại bỏ M3CP bằng cách đổi M3 destination, nhưng chưa implement.)

## 5.7 M4: x\_proj linear (d\_inner → dt\_rank + 2 × d\_state)

### PyTorch

```
self.x_proj = nn.Linear(d_inner=128, dt_rank + 2 × d_state, bias=False)
# Output: vector chiều (dt_rank + 2 × d_state) per timestep
# Cho block 0: dt_rank=4, d_state=16 → output dim = 4 + 32 = 36
# Pad lên 48 (= 3 × 16) cho phân chia 16-lane.
# Output split: [delta_raw (4), B_raw (16), C_raw (16)]
```

### FPGA

x\_proj tổ chức weight thành 3 group × 16 lane:

- Group 0: W cho delta (4 valid + 12 zero pad)
- Group 1: W cho B (16 lane)
- Group 2: W cho C (16 lane)

MAC qua d\_inner = 128 input channels, 3 group output:

```
for t in 0..T-1:
  for c_grp_out_xp in 0..2: # 3 nhóm
    for mac_idx in 0..127:
      MAC tương tự P1, với pe_A = u_safe[mac_idx, t], pe_B = W_xp[c_grp×16+lane,
      mac_idx]
      write xproj_out[c_grp_out_xp × 16..+15, t] to B_X_CONV + t × 3 + c_grp_out_xp
```

**B\_X\_CONV được reuse** ở đây cho x\_proj output (vì x\_conv đã consumed ở M3). Compact memory map cho phép.

Cycle M4 / timestep:  $3 \times (128 \times 3 + 3) = 1161$  cycle. Block 0 total:  $\sim 1.16M$ .

## 5.8 M5: dt\_proj + bias + softplus

### PyTorch

```
self.dt_proj = nn.Linear(dt_rank=4, d_inner=128)
delta_raw = ... # (B, T, 4) từ x_proj
delta = F.softplus(dt_proj(delta_raw) + dt_proj.bias) # (B, T, 128)
```

### FPGA

Linear 4 → 128 per timestep, sau đó softplus per element.

```
for t in 0..T-1:
  for c_grp_m in 0..7:
    for mac_idx in 0..3: # dt_rank = 4 input
      MAC với pe_A = delta_raw[mac_idx, t], pe_B = W_dt[c_grp_m × 16 + lane,
mac_idx]
      # PE_out = linear projected delta (16 channel × 16-bit, registered)
      # Apply softplus per lane via sp LUT
      sp_in[0..15] ← sat_add16(pe_out[lane], bias[lane])
      sp_o[0..15] ← Softplus_LUT(sp_in[lane])

S_M5_WRITE:
  m_wr_addr ← A_X_INNER + t_stride_m + c_grp_m # GHI ĐỀ x_inner / silu
  m_wr_data ← {sp_o[15..0]}
  bank_sel ← 1
```

→ Sau M5, A\_X\_INNER chứa delta thay vì u. Đó là lý do M3CP đã copy u sang B\_U\_SAFE trước đó.

Cycle M5 / timestep: ~50. Block 0 total: ~50K.

## 5.9 M6A: tính $dA = \exp(\delta_t \times A)$ per channel + read B

### PyTorch (concept)

```
# A = -exp(A_log), shape (d_inner, d_state) = (128, 16)
# delta = từ M5, shape (B, T, d_inner) = (1, T, 128)
# Per (t, c, s): dA[t, c, s] = exp(delta[t, c] × A[c, s])
# Per (t, s): B[t, s] = x_proj output (đã có ở B_X_CONV)
```

### FPGA

S\_M6A có 16 substep × s\_idx (cho mỗi state dimension). Mỗi substep:

```

S_M6A_DA_READ: read A[c_grp_m × 16..+15, s_idx], read delta[c_grp_m × 16..+15, t]
S_M6A_DA_COMP: tính delta × A per lane qua PE Array MUL mode (a_is_vector=1)
S_M6A_DA_LUT: exp_in[lane] = sat result; exp_o[lane] = Exp_LUT(exp_in[lane])
S_M6A_DA_LATCH: dA_reg[lane] ← exp_o[lane]
S_M6A_T2_READ: read u_safe[c_grp_m × 16..+15, t] from B_U_SAFE
S_M6A_T2_LATCH: u_scalar_reg[lane] ← u
S_M6A_DB_READ: read B[s_idx, t] (scalar) → B_scalar_reg
S_M6A_DB_LATCH: term2_reg[lane] = (B_scalar_reg × delta[lane]) × u
                    (per lane MUL combinational)

```

Đây là phần phức tạp nhất Mamba. Tổng cycle / (t, c\_grp\_m, s\_idx): ~15 cycle.

5.10 M6B:  $\text{scan } h_t = dA \times h_{\{t-1\}} + dB \times u + (\text{cập nhật } h_{\text{reg}})$

### PyTorch

```

# h_0 = 0; for t in 0..T-1: for c, s: h[t, c, s] = dA[t,c,s] × h[t-1,c,s] +
dB[t,c,s] × u[t,c]
# Đây là vòng lặp sequential (h phụ thuộc h trước)

```

### FPGA

State **h\_reg** (256-bit) lưu hidden state cho 16 lane (= 1 c\_grp\_m × 16 state dim). Vòng lặp:

```

for t in 0..T-1:
  for c_grp_m in 0..7:
    h_reg ← 0 (chỉ ở t=0)
    for s_idx in 0..15:
      đọc dA_reg, dBxu từ M6A
      h_reg[lane] ← sat_add16(dA_reg[lane] × h_reg[lane], term2_reg[lane])
        (PE Array MUL + ADD; 1 cycle reduction per lane)
      write h_reg[lane] to A_H_STATE[c_grp_m × 16 + s_idx]

```

Cycle / (t, c\_grp\_m, s\_idx): ~17. Block 0:  $1000 \times 8 \times 16 \times 17 \approx 2.17\text{M}$  cycle — **lớn nhất trong Mamba**.

5.11 M7:  $y_{\text{gated}} = (C \times h + D \times u) \times \text{SiLU}(z_{\text{gate}})$

PyTorch:

```

y = (h × C^T).sum(dim=state) + D × u    # SSM output
y_gated = y × silu(z)

```

FPGA: tương tự M6B nhưng read C, h, D, u → tính y → silu(z) → multiply. Cycle cao do nhiều substep.

5.12 M8: out\_proj (linear d\_inner → d\_in = 64)

Cuối Mamba: project y\_gated từ 128 channel xuống 64.

```
mamba_out = self.out_proj(y_gated)
```

FPGA MAC giống M1A nhưng input d\_inner = 128, output d\_in = 64. Ghi vào A\_MAMBA\_OUT.

Cycle M8 / t: ~800. Block 0 total: ~800K cycle.

5.13 Tổng cycle phase Mamba block 0

| Sub-phase  | Cycle (block 0, T=1000) |
|------------|-------------------------|
| NORM + M1A | 1.6M                    |
| NORM + M1B | 1.6M                    |
| M2         | 96K                     |
| M3         | 40K                     |
| M3CP       | 40K                     |
| M4         | 1.16M                   |
| M5         | 50K                     |
| M6A        | 1.9M                    |
| M6B        | 2.17M                   |
| M7         | 1.1M                    |
| M8         | 800K                    |
| Total      | ~10.5M cycle            |

Sau khi M8 xong:

- A\_MAMBA\_OUT (ram\_a[8000..11999]) chứa mamba\_out\_q[0..63, 0..999]
- done\_mamba ← 1
- state → S\_FIN\_READ

6. Phase 4 — FIN (Final stage Block 0)

6.1 PyTorch

```
# inception_out (cat 4 branches + BN + ReLU) đã trong incep_cat_q
x1 = self.relu(self.bn(incep_cat)) # bn_relu(inc) per channel
```

```
x2 = self.relu(mamba_out)      # relu(mamba_out)
out = x1 + x2                  # sat_add
```

**bn\_relu** per channel:

```
bn_out = sat16(sat16(inc × scale >> FB) + shift)
x1      = relu(bn_out) = max(bn_out, 0)
```

## 6.2 FPGA — flow

```
for t in 0..T-1:
  for c_grp_out in 0..3:  # 4 nhóm × 16 channel = 64 output
    S_FIN_READ:  đọc inception output cho c_grp_out (từ A_CH1, B_CH2, B_CH3,
    B_CH4
                  tùy fin_branch)
    S_FIN_MUL:   đọc Inc BN scale, shift; cache incep_reg ← m_rd_data
    S_FIN_WAIT2: đọc mamba_out (cùng địa chỉ ở bank A)
    S_FIN_WRITE: ghi m_wr_data, mỗi lane = sat_add16(bn_relu(incep, scale,
    shift),
                  relu16(mamba))
    m_wr_addr = (c_grp == 0) ? B_FINAL_OUT + t_stride_out
                  : A_FINAL_OUT + t_stride_out + c_grp
    bank_sel = (c_grp == 0) ? 0 : 1
    S_FIN_NEXT: advance t hoặc c_grp; cuối cùng → CASCADE/HEAD/DONE
```

## 6.3 Cycle FIN block 0: ~10K (negligible)

## 6.4 Memory sau FIN

- **B\_FINAL\_OUT[t × 4]** (c\_grp=0): block 0 output cho channel 0..15 mỗi t
- **A\_FINAL\_OUT[t × 4 + c\_grp]** (c\_grp=1..3): output cho channel 16..63

# 7. Phase 5 — Cascade Block 0 → Block 1

## 7.1 PyTorch

Block 0 output (1, 64, 1000) trực tiếp là input của Block 1 (không qua maxpool vì giữa block 0 và 1 không có maxpool).

## 7.2 FPGA — copy FINAL\_OUT → A\_INPUT\_BASE

```
S_CASCADE_RA: bank_sel ← (c_grp == 0) ? 1 : 0
               m_rd_addr ← (c_grp == 0 ? B_FINAL_OUT : A_FINAL_OUT) +
                           src_t_a × CH_OUT + c_grp
S_CASCADE_WA: wait
```

```

S_CASCADE_RB: latch m_rd_data → max_buf
               nếu need_pool: set m_rd_addr cho src_t_b
S_CASCADE_WB: wait
S_CASCADE_WR: m_we ← 1; bank_sel ← 1 (write ram_a)
               m_wr_addr ← A_INPUT_BASE + t_out × CH_OUT + c_grp
               m_wr_data ← need_pool ? elem_max16(max_buf, m_rd_data) : max_buf
               advance t_out; cuối cùng done_all ← 1

```

Block 0 → 1: cascade\_mode = 1, need\_pool = 0 → copy mode.

Cost:  $4 \text{ c\_grp} \times 1000 \text{ t} \times 5 \text{ cycle} \approx 20\text{K cycle}$ .

Sau cascade, ram\_a[0..3999] chứa block 0 output, sẵn sàng làm input cho Block 1.

### 7.3 done\_all pulse + chờ start mới

Host detect **done\_all** = 1, DMA-load block 1 weights, pulse **start**. Lần này:

- enc\_mode = 0 (encoder đã chạy lần đầu)
- cascade\_mode = 1, need\_pool = 0 (block 1 → 2 không cần pool — nhưng block 1 → 2 phải pool!)

**Sửa:** cascade/need\_pool áp dụng KHI block hiện tại xong (chứ không phải khi block sau chạy). Vậy:

- Block 0 (đang chạy): cascade=1, need\_pool=0 → sau xong copy thẳng cho block 1
- Block 1 (sẽ chạy sau): cascade=1, need\_pool=1 → sau xong pool stride-2 cho block 2
- Block 2: cascade=1, need\_pool=0
- Block 3: cascade=1, need\_pool=1 → pool cho block 4
- Block 4: cascade=0, head\_mode=1

## 8. Block 1 — lặp giống Block 0

Sau cascade block 0, host trigger Block 1 với cùng config (CH\_IN=CH\_OUT=4, CH\_M=8, DT\_RANK=4, T=1000). Block 1 weights mới được DMA-load.

Flow giống Block 0:

- S\_IDLE start → enc\_mode=0 → S\_P1\_MAC trực tiếp
- P1 → BR → NORM/Mamba → FIN
- Cuối FIN: cascade\_mode=1, need\_pool=1 → S\_CASCADE với pool

Cycle Block 1 ≈ Block 0 = ~15.5M cycle.

### 8.1 Cascade Block 1 → 2 với pool

**need\_pool\_reg** = 1. Cascade FSM đọc 2 timestep liên tiếp, lấy max:

```

S_CASCADE_RA (t_out): m_rd_addr ← src for src_t_a = 2 × t_out
S_CASCADE_RB:         max_buf ← m_rd_data;
                       m_rd_addr ← src for src_t_b = 2 × t_out + 1

```

```
S_CASCADE_WR:      m_wr_data ← elem_max16(max_buf, m_rd_data) (16 lane max)
                   m_wr_addr ← A_INPUT_BASE + t_out × 4 + c_grp
```

Sau pool, A\_INPUT\_BASE chứa 500 timestep (thay vì 1000).

## 9. Block 2, 3 — T = 500

Tương tự Block 0/1 nhưng T = 500. Host set T\_MAX = 500 trước start.

Cycle / block 2: ~7.5M. Cycle / block 3: ~7.5M.

Cascade Block 3 → 4 với pool → T = 250.

## 10. Block 4 — T = 250, d\_out = 128, d\_inner = 256

### 10.1 Khác biệt config

- CH\_OUT = 8 → d\_out = 128 (gấp đôi)
- CH\_M = 16 → d\_inner = 256 (gấp đôi)
- DT\_RANK = 8
- T = 250

→ Inception có thêm **c\_grp\_br** loop (br\_grp\_last = 1, br\_dim\_groups = 2). MAC qua dim = 32 thay vì 16. Mamba scan qua 16 c\_grp\_m thay vì 8.

### 10.2 Cycle Block 4

| Phase        | Cycle        |
|--------------|--------------|
| Phase 1      | 390K         |
| Inception    | 3.86M        |
| Mamba        | 7.43M        |
| FIN          | 16K          |
| <b>Total</b> | <b>11.7M</b> |

Cuối FIN với **cascade\_mode=0**, **head\_mode=1**:

```
S_FIN_NEXT khi c_grp == ch_out_last (= 7) và t_cnt == t_last (= 249):
  if (cascade_mode_reg) { ... }
  else if (head_mode_reg) {
    gap_c_grp ← 0; gap_t ← 0
    for i in 0..15: gap_sum[0..7][i] ← 0
    state ← S_GAP_READ
  }
```

→ Chuyển sang GAP.

## 11. Phase 6 — GAP (Global Average Pooling)

### 11.1 PyTorch

```
x = x.mean(dim=-1)    # (B, 128, T=250) → (B, 128)
```

Per channel  $c$ :  $\text{gap}[c] = (1/T) \times \sum_{t=0..T-1} \text{final}[c, t]$

### 11.2 FPGA — sum + multiply by reciprocal

```
for c_grp_in 0..7:
  for t_gap 0..249:
    bank_sel ← (c_grp_in == 0) ? 1 : 0    # đọc B_FINAL_OUT hoặc A_FINAL_OUT
    m_rd_addr ← base + t_gap × CH_OUT + c_grp_in
    đợi BRAM read 1 cycle
    for each lane i in 0..15:
      gap_sum[c_grp_in][i] += sign_extend(m_rd_data[i × 16 +: 16] → 24-bit)
```

Sau khi hoàn thành accumulate ( $8 \times 250 = 2000$  cycle MAC +  $\sim 1000$  cycle wait =  $\sim 3000$  cycle), finalize:

```
for c_grp 0..7:
  for lane 0..15:
    gap_q_reg[c_grp][lane × 16 +: 16] ← sat_mul_q15(gap_sum[c_grp][lane])
    # sat_mul_q15 = sat16( gap_sum × INV_T_Q15 >> 15 )
    # INV_T_Q15 = 131 (≈ 215 / 250)
```

`sat_mul_q15` per lane: 1 cycle combinational (1 DSP mul  $24 \times 16 \rightarrow 40$ -bit, shift

15, sat16). 8 cycle cho 8 `c_grp`.

Sau GAP: `gap_q_reg[0..7]` chứa 128 GAP values (16 lane  $\times$  8 word).

### 11.3 Trace S\_GAP\_LATCH chi tiết (gap\_c\_grp=0, gap\_t=0)

```
m_rd_data tại cycle này = ram_b[B_FINAL_OUT + 0 × 8 + 0] = block4_final[0..15, 0]
gap_sum[0][0] ← gap_sum[0][0] + sign_extend(m_rd_data[0 +: 16], 24)
gap_sum[0][1] ← gap_sum[0][1] + sign_extend(m_rd_data[16 +: 16], 24)
...
gap_sum[0][15] ← gap_sum[0][15] + sign_extend(m_rd_data[240 +: 16], 24)
state ← S_GAP_NEXT
```

Combinational 16-input add per lane: nhẹ, không là CP.

## 11.4 done\_gap pulse

Cuối S\_GAP\_FIN\_NEXT (sau khi gap\_q\_reg cuối cùng được tính):

```
done_gap ← 1
fc_class ← 0; fc_grp_in ← 0; fc_lane ← 0; fc_acc ← 0
c_rd_addr ← C_FC_BIAS    // = 68
state ← S_FC_LOAD_BIAS
```

## 12. Phase 7 — FC (Classifier)

### 12.1 PyTorch

```
self.classifier = nn.Linear(2 × d_model = 128, n_classes = 5, bias=True)
logits = self.classifier(gap)
# logits[c] = Σ_i W_fc[c, i] × gap[i] + b_fc[c]
```

### 12.2 FPGA — lane-serial MAC (sau fix timing)

#### S\_FC\_LOAD\_BIAS → S\_FC\_BIAS\_WAIT

```
S_FC_LOAD_BIAS:    state ← S_FC_BIAS_WAIT  (1 cycle wait BRAM)
S_FC_BIAS_WAIT:
    fc_bias_lane[0] ← c_rd_data[0 +: 16]    // bias class 0
    fc_bias_lane[1] ← c_rd_data[16 +: 16]   // bias class 1
    ...
    fc_bias_lane[4] ← c_rd_data[64 +: 16]   // bias class 4
    w_rd_addr ← W_FC_BASE + fc_class × 8 + fc_grp_in = 14064 + 0 + 0 = 14064
    fc_acc ← 0
    substep ← 0
    state ← S_FC_MAC
```

#### S\_FC\_MAC chi tiết (fc\_class=0, fc\_grp\_in=0)

```
substep 0:
    w_rd_addr ← W_FC_BASE + 0 × 8 + 0 = 14064
    fc_gap_word ← gap_q_reg[fc_grp_in] = gap_q_reg[0]
    fc_lane ← 0
    substep ← 1

substep 1: wait BRAM 1 cycle
    w_rd_data sẽ valid ở substep 2 đầu

substep 2 (lane 0):
```

```

    fc_acc ← fc_acc + (w_rd_data[0 +: 16] × fc_gap_word[0 +: 16])
    fc_lane ← 1
    (stay at substep 2)

substep 2 (lane 1):
    fc_acc ← fc_acc + (w_rd_data[16 +: 16] × fc_gap_word[16 +: 16])
    fc_lane ← 2
    ...

substep 2 (lane 15):
    fc_acc ← fc_acc + (w_rd_data[240 +: 16] × fc_gap_word[240 +: 16])
    fc_lane ← 0
    if (fc_grp_in == 7) state ← S_FC_WAIT
    else { fc_grp_in ← 1; substep ← 0 }

```

Sau 16 cycle ở substep 2, `fc_acc` accumulated 16 product. Total per `grp_in` = 18 cycle (2 wait + 16 MAC). Per class =  $8 \times 18 = 144$  cycle. Per FC phase =  $5 \times 144 = 720$  cycle.

## S\_FC\_NEXT\_CLASS

```

case (fc_class)
    0: logit0 ← sat_add16(sat16(fc_acc >>> 11), fc_bias_lane[0])
    1: logit1 ← sat_add16(sat16(fc_acc >>> 11), fc_bias_lane[1])
    ...
endcase
if (fc_class == 4) state ← S_FC_FINALIZE
else {
    fc_class ← fc_class + 1
    fc_grp_in ← 0
    fc_acc ← 0
    substep ← 0
    state ← S_FC_MAC
}

```

## S\_FC\_FINALIZE → S\_FC\_DONE → S\_IDLE

```

S_FC_FINALIZE: done_fc ← 1; state ← S_FC_DONE
S_FC_DONE: done_phase1 ← 1; done_inception ← 1; done_mamba ← 1; done_all ← 1
            state ← S_IDLE

```

Tại đây `logit0..4` đã chứa 5 giá trị Q4.11 cuối cùng. Host detect `done_fc` và đọc.

# 13. Output: Argmax và class prediction

## 13.1 PyTorch

```
pred = torch.argmax(logits, dim=-1).item()
```

### 13.2 FPGA / Host

logit0..4 (5 × 16-bit signed) là output port của top-level. Host (CPU) đọc 5 values, so sánh:

```
max_l = logit0; pred = 0
if (logit1 > max_l) { max_l = logit1; pred = 1 }
if (logit2 > max_l) { max_l = logit2; pred = 2 }
if (logit3 > max_l) { max_l = logit3; pred = 3 }
if (logit4 > max_l) { max_l = logit4; pred = 4 }
return pred
```

→ pred ∈ {0, 1, 2, 3, 4} là class predict cuối cùng.

Đối với PTB-XL super-class:

- 0 = NORM (Normal)
- 1 = MI (Myocardial Infarction)
- 2 = STTC (ST/T Change)
- 3 = CD (Conduction Disturbance)
- 4 = HYP (Hypertrophy)

## 14. Tổng kết end-to-end cho 1 sample

### 14.1 Cycle count tổng

| Phase                           | Cycle       | Time @ 100 MHz |
|---------------------------------|-------------|----------------|
| Encoder                         | 64K         | 0.64 ms        |
| Block 0 (P1+BR+Mam+FIN+Cascade) | 15.5M       | 155 ms         |
| Block 1                         | 15.5M       | 155 ms         |
| Block 2 (T=500)                 | 7.7M        | 77 ms          |
| Block 3 (T=500)                 | 7.7M        | 77 ms          |
| Block 4 (T=250)                 | 11.7M       | 117 ms         |
| GAP                             | 6K          | 0.06 ms        |
| FC                              | 720         | 0.0072 ms      |
| <b>Tổng</b>                     | <b>~58M</b> | <b>~580 ms</b> |

DMA load overhead (load 5 block × ~13K word weight + waveform + encoder/FC weight first-time):

- Block weight: ~13K word × 5 × 2 cycle/word = 130K cycle ≈ 1.3 ms × 5 = 6.5 ms tổng

- Waveform:  $1000 \text{ word} \times 2 \text{ cycle} = 2\text{K cycle} = 0.02 \text{ ms}$
- Encoder/FC weight first time:  $\sim 108 \text{ word} \times 2 = 216 \text{ cycle} = 0.002 \text{ ms}$

**Total per sample @ 100 MHz:**  $\sim 590 \text{ ms}$  (compute + DMA).

## 14.2 Memory access pattern trong 1 sample

| Region                | Lần read                      | Lần write        |
|-----------------------|-------------------------------|------------------|
| ram_a (40 URAM)       | $\sim 12\text{M}$ (all phase) | $\sim 5\text{M}$ |
| ram_b (40 URAM)       | $\sim 10\text{M}$             | $\sim 4\text{M}$ |
| ram_weight (118 BRAM) | $\sim 30\text{M}$             | 0 (chỉ DMA)      |
| ram_const (LUT)       | $\sim 5\text{M}$              | 0                |

## 14.3 DSP utilization

- **16 DSP** ở PE\_Array dùng liên tục trong các phase MAC (P1, BR, M1A/B, M2, M4, M5, M6A/B, M7, M8) — duty cycle  $\sim 70\%$
- **1 DSP** ở RMSNorm\_Mul cho RMSNorm output
- **1 DSP** trong FC reduce (sau timing fix)
- **Các DSP khác** rải rác trong sat/bn\_relu chains ( $\sim 40$ )
- Tổng  $\sim 60$  DSP.

## 14.4 PyTorch $\leftrightarrow$ FPGA equivalence summary

| PyTorch op                                                | FPGA implementation                            | State                      |
|-----------------------------------------------------------|------------------------------------------------|----------------------------|
| <code>Conv1d(12,64,k=1) + BN</code>                       | MAC 12 inputs + bias, 4 c_grp_out              | S_ENC_*                    |
| <code>Conv1d(64,64,k=1) + BN</code> block conv            | MAC 64 inputs + bias, 4 c_grp_out              | S_P1_*                     |
| <code>Conv1d(d_out, d_out/4, k=1)</code><br>bottleneck    | MAC 64 inputs, no bias                         | S_BR_MAC<br>branch_id=0    |
| <code>MaxPool(k=3) + Conv1d B1</code>                     | 3-cycle max + MAC                              | S_BR_MAC<br>branch_id=1    |
| <code>Conv1d(dim, dim, k=9/19/39) B2/3/4</code>           | $k \times 16$ -input MAC qua kernel position   | S_BR_MAC<br>branch_id=2..4 |
| <code>RMSNorm</code>                                      | norm_sq + RSqrt ROM + RMSNorm_Mul              | S_NORM_M1*_*               |
| <code>Linear(64, 128) x2</code> in_proj                   | MAC qua 64 input, 8 c_grp_out                  | S_M1A_, S_M1B_             |
| <code>Conv1d(128,128,k=4, groups=128)</code><br>depthwise | element-wise mul (PE MUL mode)<br>$\times 4 k$ | S_M2_*                     |
| <code>SiLU</code>                                         | LUT lookup 16-lane                             | S_M3_*                     |
| (mem layout artifact)                                     | copy u_safe                                    | S_M3CP_*                   |

| PyTorch op                                              | FPGA implementation       | State       |
|---------------------------------------------------------|---------------------------|-------------|
| <code>Linear(128, 36) x_proj</code>                     | MAC qua 128, 3 c_grp_xp   | S_M4_*      |
| <code>Linear(4, 128) + bias + softplus</code>           | MAC + LUT                 | S_M5_*      |
| <code>exp(<math>\delta \cdot A</math>)</code>           | MUL + Exp_LUT             | S_M6A_DA_*  |
| SSM scan $h = dA \cdot h + dB \cdot u$                  | MUL + sat_add             | S_M6B_*     |
| $C \cdot h + D \cdot u$ y_gated $\times \text{silu}(z)$ | MUL + sat + LUT           | S_M7_*      |
| <code>Linear(128, 64) out_proj</code>                   | MAC qua 128               | S_M8_*      |
| <code>BN+ReLU(inc) + ReLU(mam)</code> final             | bn_relu + relu + sat_add  | S_FIN_WRITE |
| copy/pool cho block kế                                  | cascade FSM               | S_CASCADE_* |
| <code>mean(dim=-1)</code> GAP                           | accumulator + sat_mul_q15 | S_GAP_*     |
| <code>Linear(128, 5)</code> classifier                  | lane-serial MAC + bias    | S_FC_*      |
| <code>argmax(logits)</code>                             | host CPU                  | —           |

## 15. Phụ lục — Mapping cụ thể 1 phép tính từ float $\rightarrow$ integer

### 15.1 Ví dụ: encoder output channel 0, timestep 0

#### Float (PyTorch)

```

W_conv[0, :, 0] = [0.1, -0.2, 0.05, ..., -0.15] # 12 weights
b_conv[0] = 0.3
 $\gamma_{\text{BN}}[0] = 1.2$ 
 $\beta_{\text{BN}}[0] = 0.05$ 
 $\mu_{\text{BN}}[0] = 0.1$ 
 $\sigma_{\text{BN}}[0] = 1.5$ 

W_fused[0, j] = W_conv[0, j, 0]  $\times \gamma / \sigma$ 
                $\approx [0.08, -0.16, 0.04, \dots, -0.12]$ 
b_fused[0] = (b_conv[0] -  $\mu$ )  $\times \gamma / \sigma + \beta$ 
             $\approx 0.21$ 

x[0..11, 0] = [-0.5, 0.3, -0.1, 0.8, ..., -0.2] # 12 leads ở t=0

enc[0, 0] =  $\sum W_{\text{fused}}[0, j] \times x[j, 0] + b_{\text{fused}}[0]$ 
            $\approx 0.45$ 

```

#### Integer (FPGA Q4.11)

```

W_q[0, j]      = floor(W_fused[0, j]  $\times 2048$ )  $\in \{164, -328, 82, \dots, -246\}$ 
b_q[0]         = floor(0.21  $\times 2048$ ) = 430

```

```

X_q[j, 0]    = floor(x[j, 0] × 2048) ∈ {-1024, 614, -204, 1638, ..., -409}

(in PE Array, j=0..11, cycle theo cycle:)
acc_raw_0 = X_q[0,0] × W_q[0,0]          = -1024 × 164      = -167,936
acc_raw_0 += X_q[1,0] × W_q[0,1]         = 614 × -328       = -201,392
acc_raw_0 += X_q[2,0] × W_q[0,2]         = -204 × 82        = -16,728
...                                         (cộng dồn 12 lần)
acc_raw_0 (sau 12 MAC) ≈ ~920,000 (40-bit signed)

# Substep 2 last MAC mac_idx=11:
out_val_0 = sat16(acc_raw_0 >>> 11) = sat16(449) = 449

# S_ENC_WRITE: lane 0:
m_wr_data[0 +: 16] = sat_add16(pe_out[0], c_rd_data[0])
                   = sat_add16(449, 430)
                   = 879 (Q4.11) = 879/2048 ≈ 0.429

# So với float 0.45 → sai số ~0.02 (~ ±10 LSB), within tolerance.

```

→ Ngân hàng bộ nhớ ram\_a tại offset 0 lưu word có lane 0 = 879, lane 1..15 là các output channel 1..15 tương tự.

## 16. Note kết luận

Toàn bộ pipeline cho 1 sample thực hiện ~60 triệu phép MAC trên FPGA, qua ~147 FSM state, sử dụng 16 DSP cho PE\_Array cộng các DSP/LUT/CARRY rải rác, ~580 ms latency. Mọi phép tính có **ánh xạ chính xác tới một block PyTorch tương ứng** (đã verify byte-exact 100/100 sample qua TB v2). Sự sai số AUC -0.0026 so với float64 reference đến hoàn toàn từ quantization Q4.11, không phải từ design HW.

Mọi state nói trên đều **tuần tự**, không có parallelism giữa các phase (encoder phải xong mới đến P1; M1A xong mới đến M1B; v.v.). Parallelism duy nhất là **16-way lane** trong PE\_Array. Đây là trade-off có chủ ý: thiết kế tận dụng PE\_Array 100% trong các phase MAC chính, đổi lấy cycle count cao (~58M cycle/sample) nhưng resource thấp (60 DSP) và Fmax cao (100 MHz). Real time per sample = 580 ms vẫn rất khả thi cho ECG offline diagnosis.